HPPS Exam

exam id: 127

Jan 2024

Introduction

In this report, I detail my approach and findings from the performance analysis of N-body simulations, focusing on both a naive implementation and an optimized Barnes-Hut algorithm variant. The benchmarking was conducted on a 2021 M1 MacBook Pro, featuring an 8-core CPU. To ensure consistency in results, each benchmark was run multiple times, balancing the need for precise data against practical constraints. Key modifications and implementations were made to nbody.c and nbody-bh.c. Efforts were concentrated on reasonable parallelization strategies, avoiding over-complexity such as parallelizing deeply nested loops. Dataset selection was pivotal, chosen to ensure sufficient workload for the processor's capabilities.

Section 1

a.

In read particles(), failures can occur firstly when opening the file. If the file does not exist or there are permission issues, 'fopen' will return NULL, which the code checks. Secondly, memory allocation might fail if the system lacks sufficient memory, as indicated by 'malloc' returning NULL. This is also checked in the code. However, the function assumes the file is correctly formatted and contains the expected amount of data. Failures in reading the correct data amount due to file corruption or unexpected file format are not currently checked.

b.

In write particles(), the initial point of failure is similar: issues in opening the file for writing. This could be due to permissions or other system-related restrictions. The code does handle this scenario. The next potential failure is during the write operation with 'fwrite'. Possible issues include disk space insufficiency or filesystem errors, which can prevent data from being written correctly. These scenarios are not handled in my implementation.

c.

The memory taken up by the result of 'read-particles() ' when it reads a particle file with n particles is primarily due to the allocation for the array of particles. Each particle consists of 7 double-precision numbers (each taking 8 bytes) and a 32-bit integer for the count of particles. The total memory used for storing the particles is therefore $56n$ bytes (since each particle takes up 56 bytes).
The peak size of memory allocated during the execution of ' read-particles() " occurs just after the memory allocation for the particles. At this point, the memory usage is the sum of the allocated particle array (which is $56n$ bytes) and any temporary memory used for file reading and function calls.
So, the peak memory usage while 'read-particles()' is executing can be estimated as slightly more than 56n bytes, with the exact amount depending on system-specific overheads.

Section 2

a.

The nbody function demonstrates good temporal locality, particularly in the inner loops where the same data elements (forces and particle properties) are accessed repeatedly in a short time span. The spatial locality is mixed. While accessing the forces array sequentially exhibits good spatial locality, the nested loop for force calculation disrupts this

by accessing non-sequential elements in the ps array. This random access pattern reduces spatial locality, especially with a large number of particles.

The use of OpenMP for parallelization may influence locality. While it enhances performance through concurrent processing, it can lead to increased cache misses as different threads access disparate memory locations, potentially reducing spatial locality. Overall, I would say the implementation shows strong temporal locality but mixed spatial locality, affected by the nested loop structure and parallelization.

b.

Main Time Step Loop (Outermost Loop):
Parallelization Potential:
This loop iterates over each time step. Parallelizing this loop is not straightforward because each time step depends on the state of particles from the previous step. Therefore, parallelizing it could lead to incorrect results due to dependencies across steps.

Force Calculation Loop (First Inner Loop):
Parallelization Potential:
This loop calculates forces for each particle independently of others. It's inherently parallelizable as the force calculation for each particle doesn't depend on the simultaneous calculation for others.

Implementation:
I've parallelized this loop using OpenMP ('#pragma omp parallel for *). I did this, as it allows the force calculations for different particles to be carried out concurrently, improving performance.

Nested Loop Inside Force Calculation:
Parallelization Potential:
This nested loop runs through all particles to accumulate forces on a single particle. Parallelizing this loop is challenging due to data dependencies and potential race conditions. Each iteration modifies the force on a specific particle, depending on all other particles. Parallelizing it requires careful handling of these dependencies and might not give significant performance gains due to the increased overhead of managing these dependencies.

Velocity and Position Update Loop (Second Inner Loop):
Parallelization Potential:
This loop updates the velocities and positions of each particle based on the calculated forces. Since each particle's update is independent of others, this loop is ideal for parallelization.

Implementation:
I have parallelized this loop using OpenMP, because this allows for concurrent updates of each particle's state, improving performance, especially for a large number of particles.

c.

Strong scaling

Strong scaling measures the program's performance improvement as the number of threads increases while keeping the problem size (number of particles) constant. The ideal scenario in strong scaling is that doubling the number of threads halves the computation time.

Below are my results for strong-scaling of the program:

Number of Threads	Number of Particles	Execution Time (seconds)
1	1000	0.009912
2	1000	0.004799
4	1000	0.002599
8	1000	0.001595

The results show a clear trend of decreasing execution time as the number of threads increases. This indicates that the program scales well with increased parallelism for a fixed problem size.

Particularly noteworthy is the near-linear speedup from 1 to 8 threads. For example, moving from 1 to 4 threads, the time approximately quarters, which is close to ideal scaling.

Weak scaling

Weak scaling measures how the execution time increases as the number of threads and the particle size increase proportionally. Ideally, in weak scaling, the execution time should remain constant as both the problem size and the number of threads grow. Below are my results for weak-scaling of the program:

Number of Threads	Number of Particles	Execution Time (seconds)
1	1000	0.009912
2	2000	0.016078
4	4000	0.028205
8	8000	0.060515

The execution time increases as both the number of particles and threads increase, indicating that the program does not exhibit ideal weak scaling. The increase in execution time is more than linear relative to the increase in problem size, suggesting that the program becomes less efficient as the workload increases. The results suggest that while the program can handle larger problems with more threads, it does so with decreasing efficiency.

Section 3

a.

Good Temporal Locality is observed as the program repeatedly accesses particle data (positions and velocities) and octree nodes (during construction and traversal) within each time step. This repeated access to the same data points demonstrates efficient temporal locality. Spatial Locality:

The program also exhibits good spatial locality since particle data is stored contiguously in memory. The Octree structure shows mixed spatial locality. While sibling nodes in the octree may be close in memory, the recursive traversal can lead to accessing distant memory locations, particularly for deeper levels of the tree or when the nodes are sparsely populated. This can reduce spatial locality, especially during the force calculation steps where different parts of the tree are accessed based on the particle distribution.

b.

The loop that iterates over the particles to find the min and max coordinates, as well as the loop that updates the velocity and position of each particle, can be parallelized since each iteration is independent of the others.

In the loop for finding min-coord and max-coord, the reduction clause is used to handle the minimum and maximum calculations in a thread-safe manner. The loop that updates particles' positions is parallelized straightforwardly with pragma omp parallel for'.

c.

Below is my comparison table for the performance of the naive N-body and the Barnes-Hut N-body implementations as the number of particles varies, along with the calculated speedup:

Number of Particles	Naive N-body Time (s)	N-body-BH Time (s)	Speedup (Naive / BH)
1000	0.010197	0.005005	2.04
2000	0.027693	0.011496	2.41
4000	0.080309	0.026330	3.05
8000	0.268303	0.054589	4.92
16000	1.016773	0.106096	9.59

The table demonstrates a consistent trend: as the number of particles increases, the Barnes-Hut algorithm provides increasing speedup compared to the naive implementation.

For lower particle counts (1000, 2000), there is a notable speedup. This improvement, although significant, is less pronounced due to the overhead associated with the Barnes-Hut algorithm, such as tree construction. As the particle count grows (4000, 8000), the speedup becomes more substantial, highlighting the efficiency gains of the Barnes-Hut algorithm in larger simulations. The most significant improvement is observed at 16000 particles, where the speedup is nearly 9.59 times. This underlines the Barnes-Hut algorithm's capability to handle large-scale simulations far more efficiently than the naive approach.

d.

Below i have made a table that shows the execution times for a fixed problem size of 4000 particles as I vary the number of threads:

Number of Threads	Number of Particles	Execution Time (seconds)
1	4000	0.029327
2	4000	0.016493
4	4000	0.006947
8	4000	0.007442

For strong scaling, the key is to keep the total problem size constant while increasing the number of threads.

I have chosen a dataset that is sufficiently large to see the benefits of parallelization, and exactly a little too large, in order to demonstrate the concept of it causing excessive memory or computational overhead, as seen in my table above, as i thought this was interesting.

There's a clear trend of decreasing execution time as the number of threads increases from 1 to 4. This indicates that the implementation benefits from parallelization, especially when moving from a single thread to multiple threads. Interestingly, the execution time slightly increases when moving from 4 to 8 threads. This could be due to several factors such as the overhead of managing more threads, contention for shared resources, or that the workload for each thread becomes too small to effectively utilize the additional threads. For this particular problem size (4000 particles), the optimal thread count appears to be around 4, beyond which the benefits of additional threads diminish.

e.

As the number of particles increases (with proportional increase in computational resources), the complexity of the Barnes-Hut algorithm escalates due to more extensive tree construction and traversal. This can lead to non-linear increases in execution time.

With larger simulations, ensuring even distribution of computational work among threads becomes challenging. The uneven distribution of particles can result in some threads having more work than others, impacting overall performance.

Practical constraints like the number of processor cores and memory capacity can limit the ability to test weak scaling effectively, especially for large-scale simulations.

The Barnes-Hut algorithm involves building an octree, whose structure and depth can vary significantly with different particle distributions. This variability can cause fluctuations in performance, complicating weak scaling measurements.

f.

Based on my benchmark results for a smaller particle count and a lower theta value, here's an analysis for a workload where n-body is faster than n-body-bh.

Benchmark Results Analysis
For the given scenario with 100 particles:
Naive N-body Simulation: 0.000082 seconds
Barnes-Hut N-body Simulation (theta = 0.05): 0.000251 seconds
The results show that the naive N-body simulation outperforms the Barnes-Hut simulation for this specific workload, which involves a smaller number of particles (100) and a lower theta value (0.05).

Interpretation
The naive implementation directly calculates forces between each pair of particles without any approximations. With only 100 particles, the number of calculations required remains within manageable limits, allowing the naive method to perform efficiently without the overhead of more complex calculations or data structures.

Setting a lower theta value in the Barnes-Hut algorithm increases its computational demand. A theta of 0.05 forces the algorithm to consider more detailed interactions between particles, approaching the accuracy of the naive method but with additional overhead from managing the octree structure. This results in longer computation times for the Barnes-Hut simulation compared to the naive approach.